Relatório Técnico - Desafio 4.0 (Projeto e Análise de Algoritmos)

Equipe LogisticaMA

5 de março de 2026

Resumo

Este documento descreve a solução desenvolvida para responder consultas do tipo *"quantos pacotes atrasaram mais de 30 minutos em uma janela de tempo"* com tempo de resposta inferior a 3 segundos, mesmo com até dois milhões de eventos. Propõe-se um índice baseado em ordenação temporal e prefix sums por hub e status, reduzindo o custo de consulta de $\Theta(n)$ para $\Theta(\log n)$. São apresentados a modelagem do problema, análise de complexidade temporal e espacial, metodologia experimental, resultados de benchmarks e conclusão sobre a conformidade com o SLA do desafio.

1 Introdução

O desafio consiste em acelerar consultas sobre logs logísticos que, na abordagem ingênua, percorrem toda a base. Em volumetrias acima de 10^6 registros, o tempo de resposta torna-se incompatível com uso operacional. O objetivo é garantir latência < 3 segundos para janelas arbitrárias, mantendo corretude dos resultados quando comparados ao baseline linear.

2 Requisitos do Desafio

- Responder consultas de atraso > 30 minutos em janelas de tempo arbitrárias com SLA de 3s até 2M de eventos.
- Permitir filtros por hub e por status (e combinações).
- Manter equivalência de resultados com a contagem linear de referência.
- Disponibilizar benchmark comparativo e dashboard exploratório.

3 Dataset e Normalização

A base oficial localiza-se em `data/logistica_ma_logs.csv`, com colunas obrigatórias {`id`, `timestamp`, `status`, `hub`, `atraso_min`}. A normalização converte timestamps para `timestamp_epoch` (int64) e cria rótulos legíveis. Ingestões CSV/JSON compartilham o mesmo contrato.

4 Solução Proposta

4.1 Pré-processamento

1. Ordenação estável por `timestamp_epoch` ($\Theta(n \log n)$) preserva ordem relativa para empates e maximiza locality de memória.

2. Construção de vetores por escopo: global, por `hub`, por `status` e por par (`hub`, `status`). Cada escopo armazena: (a) `timestamps` (int64), (b) `delays` (int32), (c) `delayed_30_prefix` (int64), (d) `delay_sum_prefix` (int64), (e) `max_delay_prefix` (int32). Custo $\Theta(n)$ por escopo, construído por agrupamento e 'cumsum' vetorial.

3. Estrutura resultante: dicionários chaveados por `hub`, por `status` e pelo par (`hub`, `status`) apontando para fatias ordenadas e prefixos compartilhados.

4.2 Consulta (como é respondida)

1. Seleção do índice: global se não houver filtro; índice do hub; índice do status; ou índice do par (hub, status) se ambos forem informados.

2. Limites temporais: busca binária via `np.searchsorted` em `timestamps` ($\Theta(\log n)$) retorna `left`, `right`. Se `left` $\geq$ `right`, retorna 0 eventos.

3. Contagem de atrasos: se `min_delay` $= 30$, usa diferença de prefixo `delayed_30_prefix[right] - delayed_30_prefix[left]` ($\Theta(1)$). Caso contrário, avalia somente o `delays[left:right]` com comparação vetorial ($\Theta(k)$, onde $k =$ `right-left`).

4. Sumário: total de eventos $= k$; atraso médio obtido por diferença de `delay_sum_prefix`; atraso máximo por 'max' na janela local; todos os passos operam apenas na fatia temporal selecionada.

5 Análise de Complexidade

5.1 Tempo

- Pré-processamento: $\Theta(n \log n)$ (ordenar) + $\Theta(n)$ (prefixos) por escopo. Em prática, construção para 2M levou ≈ 1.32s no hardware de testes.

- Consulta com `min_delay`=30: $\Theta(\log n)$ (busca) + $\Theta(1)$ (prefixo). Latência medida: $\approx 1\,\mu$s no caso de 2M (arredondamento para 6 casas no benchmark).

- Consulta com `min_delay`$\neq$30: $\Theta(\log n + k)$, onde k é o tamanho da janela temporal filtrada. Para janelas estreitas, $k \ll n$ e o custo permanece sub-milisegundo.

5.2 Espaço

- Por linha: 1 int64 (timestamp) + 1 int32 (delay) + 2 int64 de prefixo + 1 int32 de max acumulado $\approx$ 8+4+8+8+4 = 32 bytes por linha e por escopo. Como cada linha pertence a um único hub e a um único status, o fator de multiplicação dos índices se mantém próximo de 3 estruturas independentes, não uma duplicação completa de n.
- Para 2M linhas, o índice global consome $\tilde{6}$4 MB; índices por hub/status dividem os dados por grupo, mantendo a soma total na ordem de algumas centenas de MB, compatível com a máquina de teste.

6 Validação e Testes

- **Corretude de contagem**: comparação índice vs. baseline linear no dataset oficial e em amostras sintéticas.
- **Filtros**: testes unitários para filtros por hub e por status, além de `min_delay` variável.
- **Dataset**: ingestão CSV/JSON com contrato comum; dataset oficial carregado sem upload manual.
- **Benchmark CLI**: verificação automática de SLA para n = 2,000,000 e checagem de mesma resposta que o baseline.

7 Metodologia Experimental

- **Hardware**: macOS (CPU Apple Silicon, 16 GB RAM). Execuções single-threaded (numpy/pandas default) sem afinidade específica.
- **Semente**: gerador `np.random.default_rng(42)` para reprodutibilidade dos perfis sintéticos.
- **Perfis**: categorias e distribuição de `hub`, `status` e `atraso_min` extraídas da base oficial (função `load_benchmark_profile`).
- **Janelas**: para cada tamanho, escolhe-se uma janela central: `start` = índice $n/4$, `end` = $n/4 + n/2$, garantindo volume significativo e sem viés para extremos.
- **Cenários**: tamanhos {1k, 10k, 100k, 1M, 2M}; consulta com `min_delay`=30 e comparação com baseline linear. Em testes de regressão, há casos adicionais com `min_delay`=45 e filtros hub+status no dataset real.
- **SLA**: o CLI de benchmark lança exceção se o caso de 2M exceder 3s ou divergir do baseline (função `_assert_sla`).

8 Resultados e Comparativos

Resultados da execução de `uv run python benchmarks/run_benchmarks.py` (05/03/2026) no perfil sintético:

n	build (s)	linear (s)	indexado (s)	speedup (×)	mesma resposta
1,000	0.003290	0.000043	0.000001	33.57	sim
10,000	0.006147	0.000415	0.000001	334.61	sim
100,000	0.065335	0.004267	0.000001	3099.19	sim
1,000,000	0.691504	0.045138	0.000001	30802.32	sim
2,000,000	1.315253	0.089091	0.000001	62730.97	sim

Tabela 1: Benchmark sintético com perfil real. O caso de 2M satisfaz SLA de 3s (indexado na ordem de microssegundos).

Observações adicionais:

- **Speedup**: no maior cenário, o índice foi $\sim$ 62,7k vezes mais rápido que o baseline linear na mesma janela.
- **Corretude**: todas as linhas da tabela indicam respostas idênticas ao baseline; o CLI falha se houver divergência.
- **Filtros e limiares**: em testes funcionais, consultas com `hub+status` e `min_delay`=45 no dataset real retornaram valores iguais ao cálculo manual e permaneceram submilisegundo.
- **Build**: o custo de construção (1.32s para 2M) ocorre uma vez por carga; consultas subsequentes amortizam o custo.

9 Conclusões

A estrutura indexada reduz o custo de consulta de $\Theta(n)$ para $\Theta(\log n)$ com constantes pequenas: no benchmark sintético atual, as consultas ficaram na ordem de microssegundos, mesmo em 2M registros, e o SLA de 3 segundos foi atendido com ampla folga. O build em ≈ 1.3 s é pago uma vez por carga. Testes automatizados cobrem baseline linear, filtros por hub/status, limiares variados e validam que o índice replica exatamente o resultado de referência.

10 Trabalho Futuro

- Persistir índices em disco para reuso entre execuções.
- Adicionar estimativas de uso de memória por tamanho de dataset.
- Explorar paralelização na construção dos índices por hub/status.

11 Como Executar

1. Instalar dependências: `uv sync`
2. Testes: `uv run pytest`
3. Benchmarks (com SLA): `uv run python benchmarks/run_benchmarks.py`
4. Dashboard: `uv run streamlit run app/streamlit_app.py`